

Skill Week-12

1. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define NUM_JOBS 4

int main() {
    pid_t pids[NUM_JOBS];
    int status;
    time_t start, end;

    printf("Starting %d pipeline jobs...\n", NUM_JOBS);
    start = time(NULL);

    for (int i = 0; i < NUM_JOBS; i++) {
        pids[i] = fork();
        if (pids[i] < 0) {
            perror("fork");
            return 1;
        }
        if (pids[i] == 0) {
            printf("Job %d started. PID=%d\n", i + 1, getpid());
            sleep(i + 1);
            printf("Job %d completed. PID=%d\n", i + 1, getpid());
            return 0;
        }
    }

    for (int i = 0; i < NUM_JOBS; i++) {
        waitpid(pids[i], &status, 0);
        if (WIFEXITED(status) && WEXITSTATUS(status) == 0)
            printf("Job %d: SUCCESS\n", i + 1);
        else
            printf("Job %d: FAILED\n", i + 1);
    }

    end = time(NULL);
    printf("\nAll jobs completed.\n");
    printf("Total execution time: %ld seconds\n", end - start);
    printf("Pipeline stability check: PASSED\n");

    return 0;
}
```

Output:

imvarls@LAPTOP-VQ4D70L7: × + ▾

```
imvarls@LAPTOP-VQ4D70L7:~$ nano src/large_pipeline.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc src/large_pipeline.c -o src/large_pipeline
imvarls@LAPTOP-VQ4D70L7:~$ ./src/large_pipeline
Starting 4 pipeline jobs...
Job 1 started. PID=3219
Job 2 started. PID=3220
Job 3 started. PID=3221
Job 4 started. PID=3222
Job 1 completed. PID=3219
Job 1: SUCCESS
Job 2 completed. PID=3220
Job 2: SUCCESS
Job 3 completed. PID=3221
Job 3: SUCCESS
Job 4 completed. PID=3222
Job 4: SUCCESS

All jobs completed.
Total execution time: 4 seconds
Pipeline stability check: PASSED
imvarls@LAPTOP-VQ4D70L7:~$ |
```

2. To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1
# Large Pipeline Execution System

## 1. Architecture
The system contains:
User
|
v
Main Process
|-- Job 1
|-- Job 2
|-- Job 3
+-- Job 4
|
v
Wait and Monitor
|
v
Performance Report

## 2. Execution Flow
1. Main process starts.
2. Number of jobs is defined.
3. fork() creates child processes.
4. Each child performs its assigned work.
5. Parent monitors every child using waitpid().
6. Exit status is checked.
7. Execution time is calculated.
8. Final stability result is displayed.

## 3. Important System Calls
- fork(): Creates a new child process.
- waitpid(): Waits for a specific child process.
- getpid(): Returns the process ID.
- sleep(): Simulates work performed by a job.

## 4. Execution Path
Start
|
v
Create Jobs
|
v
Run Jobs
|
v
Monitor Processes
|
v
Check Exit Status
|---> Success -> Record Result
|---> Failure -> Report Failure
|
v
Measure Time
|
v
Display Final Report
|
v
End
```

```
## 5. Design Decisions
1. Multiple child processes are used to simulate multiple jobs.
2. waitpid() is used to monitor individual processes.
3. Exit status is checked to detect failures.
4. time() is used to measure execution duration.
5. The implementation is kept simple for testing process management.

## 6. Testing
The program is compiled using GCC and executed from the terminal.
Expected results:
- All jobs should start.
- All jobs should complete.
- Successful jobs should be reported.
- Total execution time should be displayed.
- Stability check should be displayed.

## 7. Conclusion
The program demonstrates process creation, process monitoring, failure detection, execution-time measurement, and basic pipeline/job stability analysis.
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + v
imvarls@LAPTOP-VQ4D70L7:~$ nano architecture.md
imvarls@LAPTOP-VQ4D70L7:~$ nano architecture.md
imvarls@LAPTOP-VQ4D70L7:~$ cat architecture.md
# Large Pipeline Execution System

## 1. Architecture
The system contains:
User
|
v
Main Process
|-- Job 1
|-- Job 2
|-- Job 3
+-- Job 4
|
v
Wait and Monitor
|
v
Performance Report

## 2. Execution Flow
1. Main process starts.
2. Number of jobs is defined.
3. fork() creates child processes.
4. Each child performs its assigned work.
5. Parent monitors every child using waitpid().
6. Exit status is checked.
7. Execution time is calculated.
8. Final stability result is displayed.

## 3. Important System Calls
- fork(): Creates a new child process.
- waitpid(): Waits for a specific child process.
- getpid(): Returns the process ID.
- sleep(): Simulates work performed by a job.

## 4. Execution Path
Start
|
v
Create Jobs
|
v
Run Jobs
|
v
Monitor Processes
|
v
Check Exit Status
|---> Success -> Record Result
|---> Failure -> Report Failure
|
v
Measure Time
|
v
Display Final Report
|
v
End

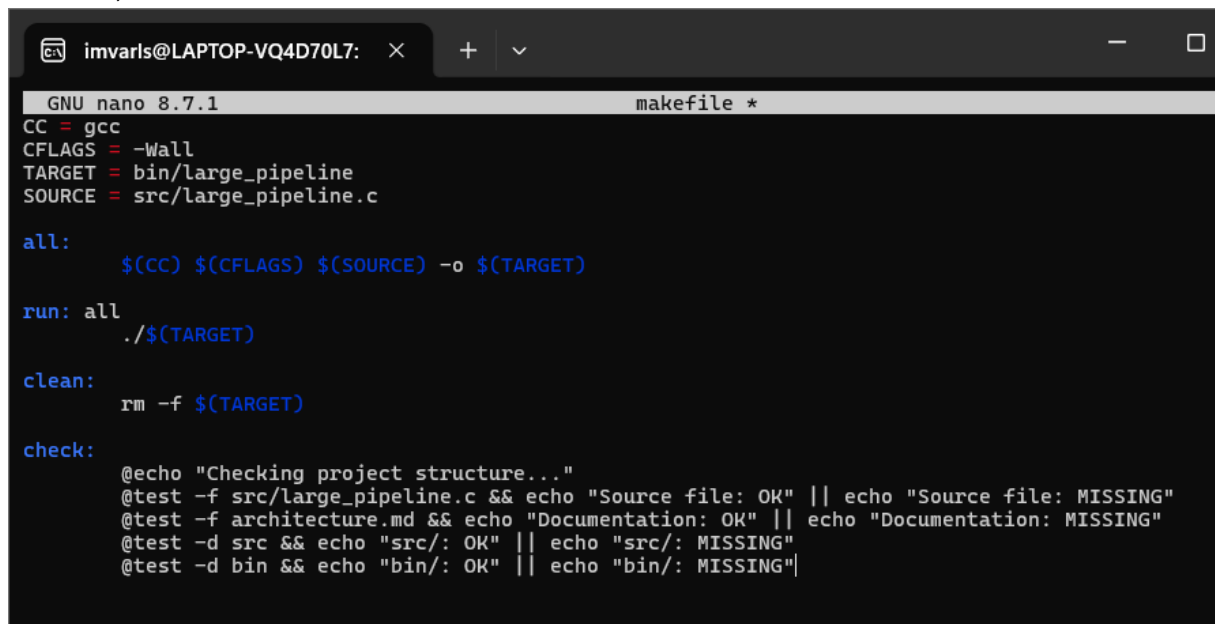
## 5. Design Decisions
1. Multiple child processes are used to simulate multiple jobs.
2. waitpid() is used to monitor individual processes.
3. Exit status is checked to detect failures.

## 5. Design Decisions
1. Multiple child processes are used to simulate multiple jobs.
2. waitpid() is used to monitor individual processes.
3. Exit status is checked to detect failures.
4. time() is used to measure execution duration.
5. The implementation is kept simple for testing process management.

## 6. Testing
The program is compiled using GCC and executed from the terminal.
Expected results:
- All jobs should start.
- All jobs should complete.
- Successful jobs should be reported.
- Total execution time should be displayed.
- Stability check should be displayed.

## 7. Conclusion
The program demonstrates process creation, process monitoring, failure detection, execution-time measurement, and basic pipeline/job sta
imvarls@LAPTOP-VQ4D70L7:~$
```

3. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables



The screenshot shows a terminal window with the title bar "imvarls@LAPTOP-VQ4D70L7: × + ▾". The terminal is running GNU nano 8.7.1, editing a file named "makefile *". The content of the Makefile is as follows:

```
GNU nano 8.7.1 makefile *
CC = gcc
CFLAGS = -Wall
TARGET = bin/large_pipeline
SOURCE = src/large_pipeline.c

all:
    $(CC) $(CFLAGS) $(SOURCE) -o $(TARGET)

run: all
    ./${TARGET}

clean:
    rm -f $(TARGET)

check:
    @echo "Checking project structure..."
    @test -f src/large_pipeline.c && echo "Source file: OK" || echo "Source file: MISSING"
    @test -f architecture.md && echo "Documentation: OK" || echo "Documentation: MISSING"
    @test -d src && echo "src/: OK" || echo "src/: MISSING"
    @test -d bin && echo "bin/: OK" || echo "bin/: MISSING"
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + v
imvarls@LAPTOP-VQ4D70L7:~$ nano makefile
imvarls@LAPTOP-VQ4D70L7:~$ make gcc -Wall src/large_pipeline.c -o bin/large_pipeline
make: *** No rule to make target 'gcc'. Stop.
imvarls@LAPTOP-VQ4D70L7:~$ make
gcc -Wall src/large_pipeline.c -o bin/large_pipeline
imvarls@LAPTOP-VQ4D70L7:~$ make run
gcc -Wall src/large_pipeline.c -o bin/large_pipeline
./bin/large_pipeline
Starting 4 pipeline jobs...
Job 1 started. PID=3352
Job 2 started. PID=3353
Job 3 started. PID=3354
Job 4 started. PID=3355
Job 1 completed. PID=3352
Job 1: SUCCESS
Job 2 completed. PID=3353
Job 2: SUCCESS
Job 3 completed. PID=3354
Job 3: SUCCESS
Job 4 completed. PID=3355
Job 4: SUCCESS

All jobs completed.
Total execution time: 4 seconds
Pipeline stability check: PASSED
imvarls@LAPTOP-VQ4D70L7:~$ make checks
make: *** No rule to make target 'checks'. Stop.
imvarls@LAPTOP-VQ4D70L7:~$ make check
Checking project structure...
Source file: OK
Documentation: OK
src/: OK
bin/: OK
imvarls@LAPTOP-VQ4D70L7:~$ |
```